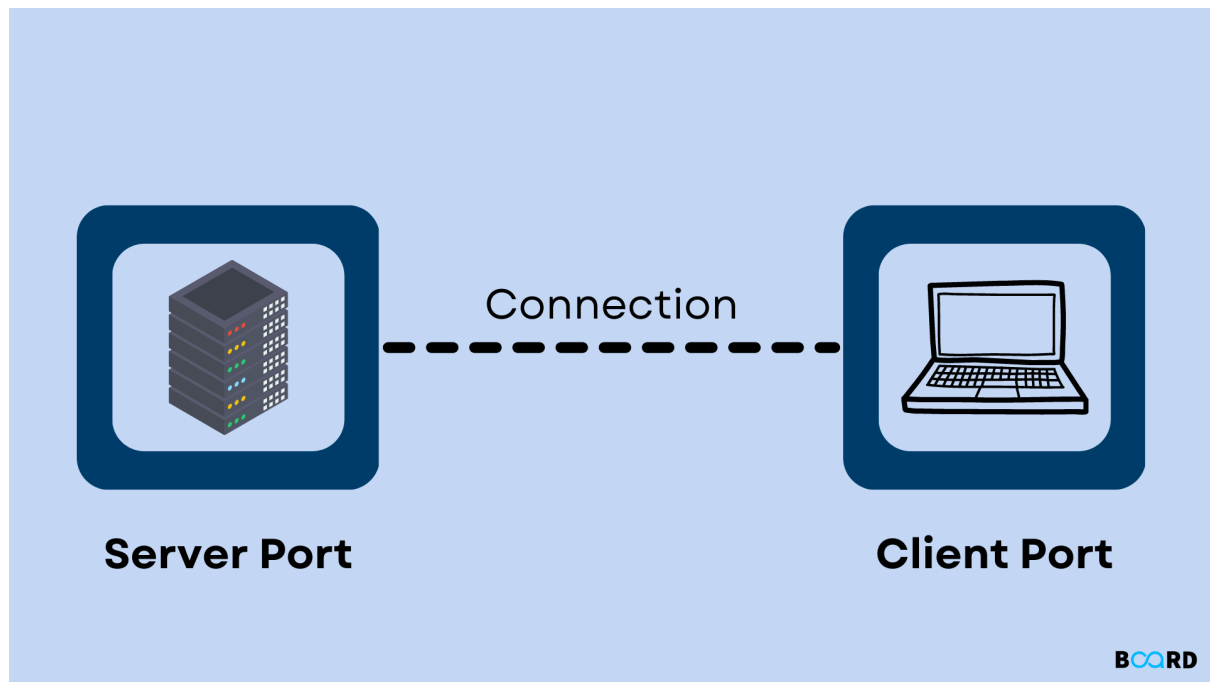


Investigación De Sockets



Centro escolar	IES Antonio Gala
Asignatura	PSP
Nombre alumno	Rafael Hidalgo Torres
Nombre profesor	Amín Harou Azouagh
Curso escolar	2ºDAM
Fecha	01-05-2026

Índice

1. El papel de los Sockets	3
¿Qué es un socket?	3
2. Uso de TCP, UDP y WebSocket	4
A. TCP (Transmission Control Protocol) - Prioridad: Fiabilidad	4
B. UDP (User Datagram Protocol) - Prioridad: Velocidad	5
C. WebSockets - Prioridad: Tiempo real en navegadores	5
Ejemplo de Caso práctico combinado:	5
3. Arquitectura y concurrencia	5
1. La arquitectura Cliente - Servidor	5
2. Concurrencia - Gestión de millones de usuarios	6
4. Seguridad y cifrado	6
1. Cifrado en tránsito (TSL/SSL)	6
2. Cifrado de extremo a extremo (E2EE)	6
3. Autenticación - ¿Quién se conecta al socket?	7
5. Problemas técnicos que resuelven estas aplicaciones	7
1. Latencia alta y red saturada	7
2. Consumo excesivo de batería en dispositivos móviles	7
3. Sincronización de estado global (“En línea”, “Escribiendo...”)	8
4. Recuperación ante caídas (Pérdida de conexión)	8

1. El papel de los Sockets

¿Qué es un socket?

Un socket es básicamente un punto final en un enlace de comunicación entre dos programas que se ejecutan a través de una red, de forma bidireccional. Es como una tubería directa que permanece abierta y conecta datos entre un móvil y los servidores de Discord, por poner un ejemplo. En esa tubería fluyen los datos en ambas direcciones de forma que no se puedan interrumpir.

Los sockets tienen un papel fundamental en plataformas como WhatsApp, Telegram o Discord, ya que su función es garantizar la inmediatez de los mensajes, las llamadas, etc, ya que, al mantener la conexión persistente el servidor no espera a que tú le preguntes las cosas, cuando alguien te manda un mensaje, simplemente el servidor empuja ese mensaje por la tubería directamente a tu pantalla en tiempo real.

En la siguiente tabla mostraré las diferencias entre las peticiones tradicionales y la conexión por Sockets:

Característica	Petición Tradicional (ej. navegar por una web)	Conexión por Sockets (ej. WhatsApp/Discord)
Flujo de datos	Unidireccional: por turnos (Cliente pide, Servidor responde).	Bidireccional: simultáneo.
Duración de la conexión	Limitada. Se abre, se envían los datos y se cierra.	Persistente. Se mantiene abierta a la espera de acciones.
Eficiencia en tiempo real	Baja. Obliga a hacer consultas constantes.	Alta. Los datos se envían al instante en cuanto existen.

2. Uso de TCP, UDP y WebSocket

Tenemos que tener en cuenta que la tubería de la que hablábamos anteriormente (el socket), está vacía, y existen unas reglas de envío que son los protocolos. Cada app usa un protocolo distinto, dependiendo de lo que estés haciendo en cada momento:

A. TCP (Transmission Control Protocol) - Prioridad: Fiabilidad

Es un protocolo orientado a la conexión que garantiza que los datos lleguen completos y en el orden exacto en el que se enviaron. Antes de enviar el mensaje real, establece una conexión segura con el servidor, para luego enumerar cada paquete de datos y hacer que el receptor confirme que lo ha recibido. Si por lo que sea uno de esos paquetes se pierde, TCP lo vuelve a enviar. Se utiliza sobre todo para mensajes de texto, o para subir archivos o fotos.

B. UDP (User Datagram Protocol) - Prioridad: Velocidad

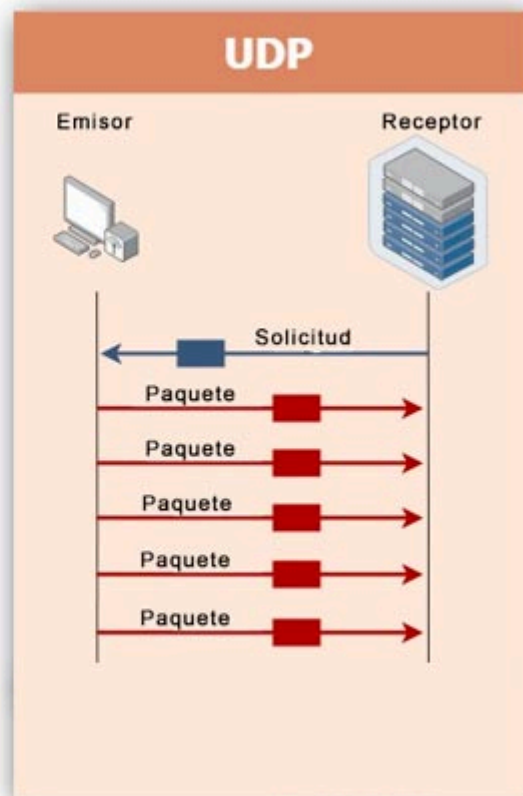
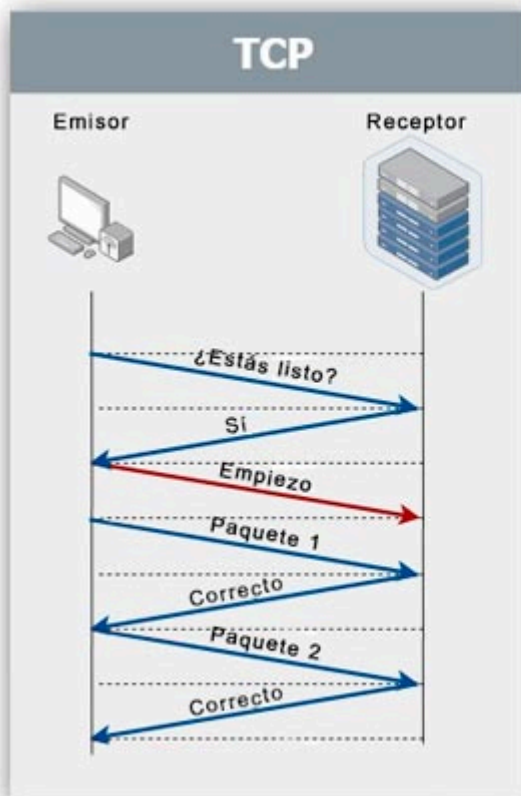
Es un protocolo que no establece conexión previa y no garantiza la entrega de los paquetes, sino que prioriza la agilidad pura. Simplemente lanza los datos hacia el receptor lo más rápido posible, no espera confirmaciones de recepción ni reenvía datos que se hayan perdido por el camino. Se usa mayormente para llamadas de voz, o videollamadas en WhatsApp o Discord, ya que en una llamada en tiempo real, es preferible perder un milisegundo de audio a que la llamada se quede congelada esperando a recuperar algún dato perdido.

C. WebSockets - Prioridad: Tiempo real en navegadores

Es un protocolo de comunicación bidireccional diseñado específicamente para superar las limitaciones de los navegadores web tradicionales. Comienza con una petición web normal, pero manteniendo una conexión abierta permanente, lo que permite que el servidor pueda lanzar información al usuario sin que este la pida. Se usa en versiones web de aplicaciones como WhatsApp o Discord, dándoles la misma inmediatez que a las aplicaciones instaladas de escritorio.

Ejemplo de Caso práctico combinado:

Cuando un usuario abre Discord, la aplicación usa WebSockets o TCP para cargar el chat y mantener el estado de “En línea”. Sin embargo, en el momento en que el usuario entra en un canal de voz, la aplicación abre una nueva ruta utilizando UDP para transmitir el audio con la menor latencia posible.



3. Arquitectura y concurrencia

1. La arquitectura Cliente - Servidor

Es un modelo de diseño de red donde las tareas y cargas de trabajo se reparten entre los diferentes servidores y clientes. Los servidores proveen los recursos y los clientes son los que piden esos recursos.

En aplicaciones como WhatsApp, Telegram o Discord, los usuarios no se comunican directamente entre sus dispositivos, ya sean móviles u ordenadores, sino que el cliente envía un mensaje a través de un socket al ordenador central de la plataforma (el servidor). Cuando el servidor recibe los datos, busca al destinatario en su base de datos y le reenvía el mensaje a través de su propio socket abierto.

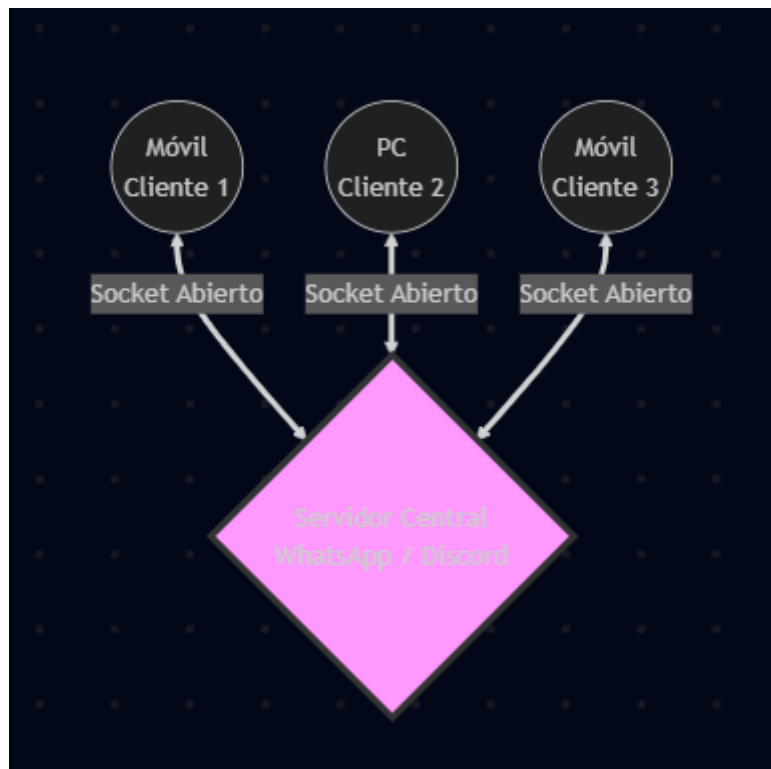
Las ventajas principales que tiene esto es que el servidor permite saber exactamente quién está en línea, guardar mensajes temporalmente al destinatario si no tiene internet en ese momento o sincronizar información en varios dispositivos a la vez, por ejemplo para usuarios que usan estas aplicaciones en el móvil y en el ordenador.

2. Concurrencia - Gestión de millones de usuarios

La concurrencia es la capacidad del servidor para manejar múltiples tareas, conexiones o procesos al mismo tiempo, sin que se bloqueen o hagan que se ralenticen unas a otras.

Si un servidor procesara los mensajes uno detrás de otro de forma secuencial (haciendo cola), el sistema colapsaría, pero gracias a la concurrencia, el servidor utiliza arquitecturas basadas en hilos de ejecución múltiples para mantener a cientos de miles de sockets abiertos a la vez, procesando y enviando datos a la vez.

Esta característica es la que salva a estas plataformas en momentos de saturación, como por ejemplo en días como Nochevieja, cuando millones de usuarios quieren mandar mensajes al mismo tiempo.



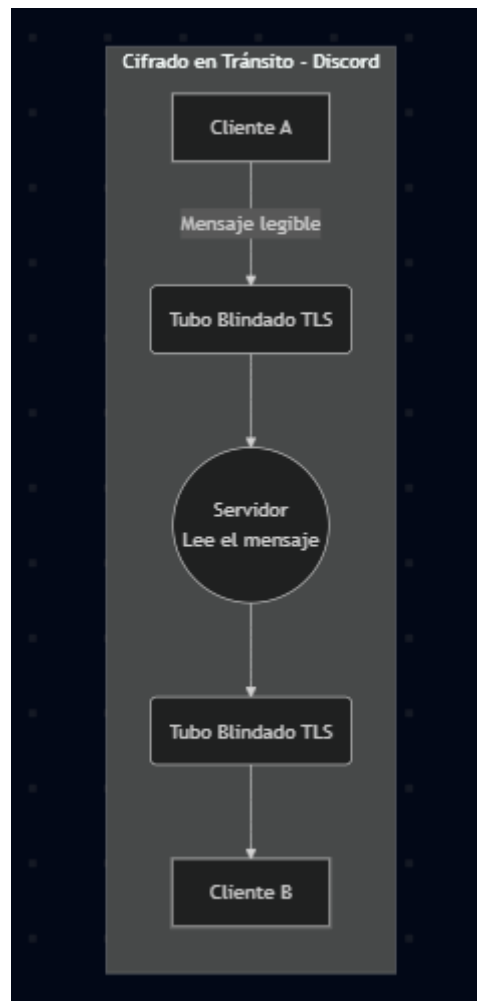
4. Seguridad y cifrado

1. Cifrado en tránsito (TLS(Transport Layer Security)/SSL(Secure Sockets Layer))

Es un protocolo que proporciona comunicaciones seguras por una red de ordenadores. Básicamente es como ponerle una capa de blindaje al socket para que nadie desde fuera pueda leer lo que está pasando en él.

Cuando el cliente y el servidor abren un socket, antes de enviar el primer mensaje realizan un proceso llamado Handshake (apretón de manos). En este proceso se acuerdan unas claves temporales, por lo que a partir de ese momento, todo lo que viaje por el socket va codificado con esa clave. Si un hacker intercepta la red wifi, solo vería un conjunto de letras sin sentido.

Discord utiliza este método por defecto para proteger la comunicación entre nuestros ordenadores y sus servidores. Aunque el mismo Discord pueda leer nuestros mensajes, nadie ajeno a la plataforma puede interceptarlo por el camino.

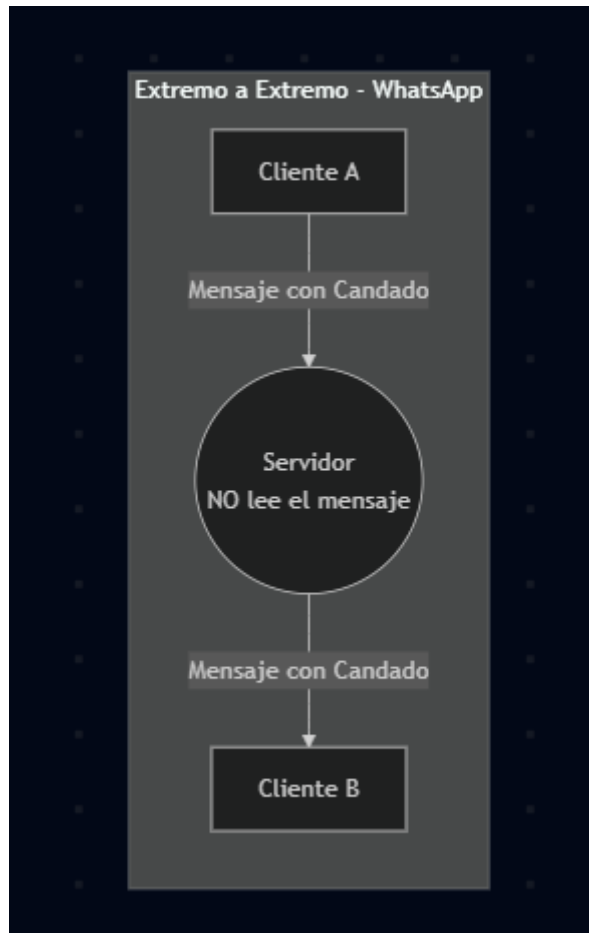


2. Cifrado de extremo a extremo (E2EE(End to End Encryption))

Es un sistema de comunicación en el que solo los usuarios que se están comunicando pueden leer los mensajes, nadie más, ni siquiera la empresa dueña del servidor tiene claves para descifrarlos.

Las claves de cifrado y descifrado se generan y se guardan solamente en los dispositivos físicos de los usuarios, es decir, en nuestros móviles u ordenadores, y cuando enviamos un mensaje, se cifra en nuestros móviles y viaja por el socket hasta el servidor, el cuál lo enruta estando cifrado, sin saber qué pone en el mensaje, y sólo se descifra cuando llega al móvil de la otra persona.

WhatsApp es el ejemplo más famoso, ya que usa el Protocolo Signal (E2EE) por defecto en todos sus chats.



3. Autenticación - ¿Quién se conecta al socket?

Es el proceso en el que se verifica que el usuario que intenta abrir un socket con el servidor realmente es quien dice ser.

Normalmente, el cliente primero debe enviar unas credenciales (usuario y contraseña) mediante una petición segura. Si el servidor valida que son correctas, autoriza la apertura del socket y vincula la conexión.

Sin autenticación, cualquier persona podría inyectar mensajes en el sistema fingiendo ser otro usuario o simplemente conectándose al puerto abierto del servidor.

5. Problemas técnicos que resuelven estas aplicaciones

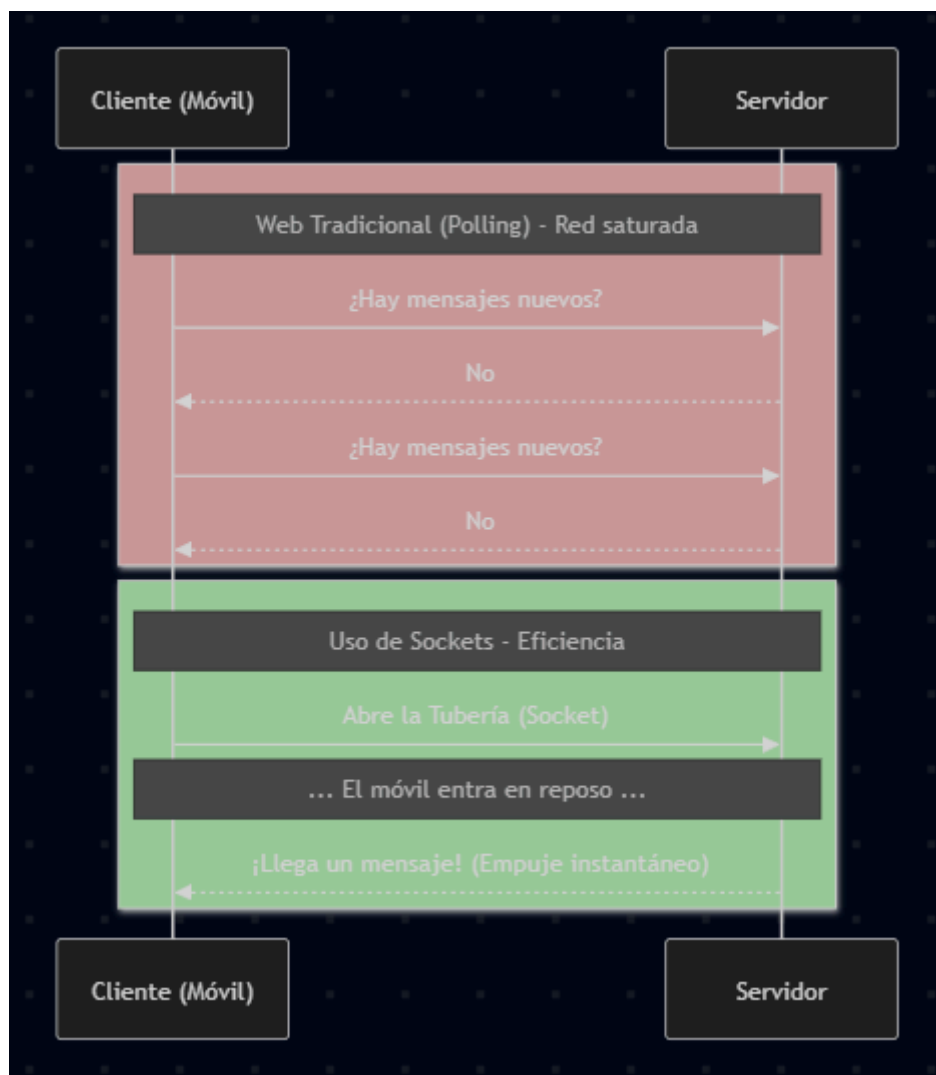
Las aplicaciones de mensajería no solo se crearon con una interfaz bonita, sino que tuvieron que resolver problemas de ingeniería de redes muy complejos para poder existir. Los problemas técnicos principales que resuelven son:

1. Latencia alta y red saturada

En la web tradicional, para saber si hay información nueva, el cliente tenía que preguntarle al servidor constantemente. Esa técnica se llamaba Polling, y generaba latencia y saturaba los servidores con peticiones vacías.

La solución fue la implementación de Sockets y WebSockets, ya que crearon la tubería que habíamos comentado antes, que siempre permanece abierta. El cliente ya no tiene que seguir preguntando, en su lugar, el servidor simplemente lanza los mensajes por la tubería en el milisegundo exacto en que lo recibe.

Esto es lo que permite la comunicación en tiempo real en aplicaciones como WhatsApp, Telegram o Discord.



2. Consumo excesivo de batería en dispositivos móviles

Si el móvil tuviera que abrir y cerrar conexiones cada pocos segundos para buscar mensajes nuevos, la antena del móvil estaría siempre activa y la batería duraría apenas un par de horas.

La solución fue mantener un único socket abierto que permanece dormido en segundo plano y que consume una fracción de la energía que consumiría hacer cientos de peticiones HTTP clásicas.

WhatsApp resuelve esto delegando también en sistemas operativos como Android o iOS, ya que la aplicación mantiene una conexión de bajo consumo que permite recibir una notificación que despierta a la aplicación solo cuando realmente ha llegado un mensaje.

3. Sincronización de estado global (“En línea”, “Escribiendo...”)

El uso de una arquitectura Cliente - Servidor centralizada con sockets fue la solución para que las aplicaciones pudieran informar a sus millones de usuarios de cuando les llega un mensaje, cuando alguien está conectado, cuando alguien está escribiendo, etc.

El servidor actúa como un gran mapa en tiempo real, cuando alguien empieza a pulsar teclas, el cliente manda un pequeño evento por el socket al servidor, y así el servidor sabe con quién estás hablando y le reenvía ese evento por su socket.

Esto es lo que permite que en WhatsApp o Discord podamos ver cuando alguien está en línea o ausente, o cuando alguien nos está escribiendo en un momento exacto.

4. Recuperación ante caídas (Pérdida de conexión)

Las redes móviles son inestables, si entras a un túnel, pierdes la conexión. Si el sistema fuera rígido, los mensajes enviados en un momento como ese se perderían en un limbo. Pero al diseñar arquitecturas pensadas para manejar la intermitencia, el servidor detecta que un socket se ha cerrado abruptamente, y guarda temporalmente los mensajes entrantes en una base de datos.

Así, cuando sales del túnel y vuelves a tener conexión, WhatsApp vuelve a abrir el socket, el servidor reconoce tu identidad, e inyecta de golpe todos los mensajes que se habían quedado en cola mientras no tenías señal.